



汇编语言

8 数据处理类指令应用与分支结构

张青

ieqzhang@zzu.edu.cn

实验二 数据表示与存储

- 1、结合教材例 2-14 程序 eg0214.c 和 eg0215.c，编写C语言程序 exp0201.c（将eg0214.c和eg0215.c两个程序整合为一个新的程序 exp0201.c），编译生成汇编语言文件 exp0201.s 和可执行程序 exp0201，记录 exp0201.s 的内容，用 GDB 调试可执行程序 exp0201，观察变量的存储。
- 2、录入教材例 2-16 程序 eg0216.c，编译生成汇编语言文件 eg0216.s 和可执行程序eg0216，记录 eg0216.s 的内容，用 GDB 调试可执行程序 eg0216，观察变量的存储。
- 3、录入教材例 2-17 程序 eg0217.c，编译生成汇编语言文件 eg0217.s 和可执行程序eg0217，记录 eg0217.s 的内容，用 GDB 调试可执行程序 eg0217，观察变量的存储。

GDB调试



- 为方便调试，应在编译时添加 -g 选项生成调试信息

```
> gcc -S -Og -g -masm=intel -o exp0201.s exp0201.c
```

```
> gcc -g -o exp0201 exp0201.s
```

启动gdb:

```
> gdb exp0201
```

常用的gdb命令:

(gdb) b main	#在main处设置断点
(gdb) r	#运行程序，至断点处或结束
(gdb) p x	#查看变量x的值
(gdb) p &x	#查看变量x的地址
(gdb) p/x \$rax	#查看寄存器rax的值，以十六进制输出
(gdb) x/4b &x	#查看变量x存储单元的内容，显示4个字节
(gdb) nexti	#单步执行汇编指令，跳过函数
(gdb) c	#继续运行到下一个断点或程序结束

请参看附录D GDB调试器

实验二 数据表示与存储

4、按照如下输出格式，在屏幕上显示 ASCII 表：

表格最上一行的数字是对应列 ASCII 代码值的低 4 位（用十六进制形式），而表格左边的数字对应行 ASCII 代码值的高 4 位（用十六进制形式）。

编写汇编语言程序exp0204.s，在数据段直接构造表格、填写相应 ASCII 代码值（不是字符本身），然后使用字符串显示子程序 DISPMSG 实现显示。

```
10 | 0 1 2 3 4 5 6 7 8 9 A B C D E F
-- + -----
20 |  ! " # $ % & ' ( ) * + , - . /
30 | 0 1 2 3 4 5 6 7 8 9 : ; < = > ?
40 | @ A B C D E F G H I J K L M N O
50 | P Q R S T U V W X Y Z [ \ ] ^ _
60 | ` a b c d e f g h i j k l m n o
70 | p q r s t u v w x y z { | } ~
```

- 数据处理类应用程序

教材章节：4.5

慕课：5.6，6.10，6.11

- ▶ 处理器识别
- ▶ 查看状态标志
- ▶ 二进制值转换成十六进制字符串

- 转移指令
- 分支结构设计

教材章节：5.1

慕课：7

- ▶ 指令寻址方式
- ▶ 无条件转移指令
- ▶ 条件转移指令
- ▶ 设置条件码

学习目标

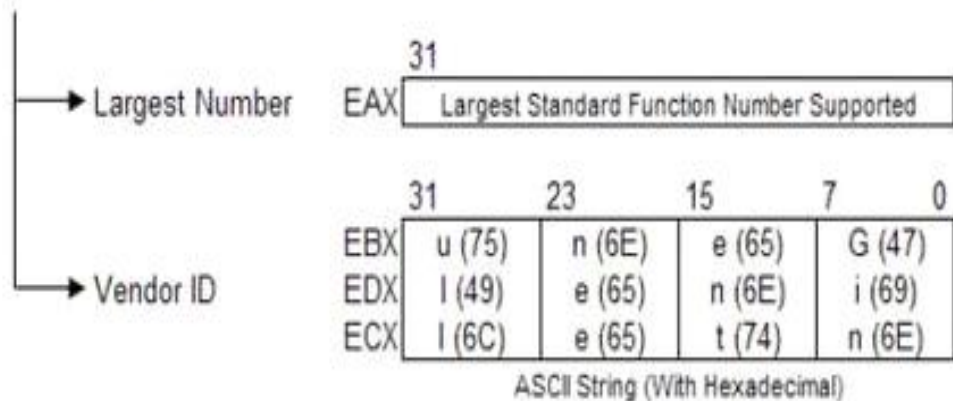


- 能说出指令的寻址方式
- 能说出转移指令的功能和格式
- 会正确判断转移发生的条件
- 会正确使用转移指令实现单分支和双分支结构
- 会阅读分析分支结构程序的功能、清楚运行结果
- 会用gdb查看寄存器和内存
- 能说出程序开发过程中出现问题的原因并解决
- 养成良好的编程习惯，养成细心、严谨的工作态度
- 养成团队合作，互相帮助的精神

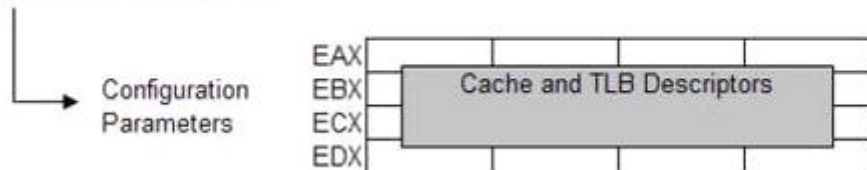
〔例4-19〕 处理器识别程序



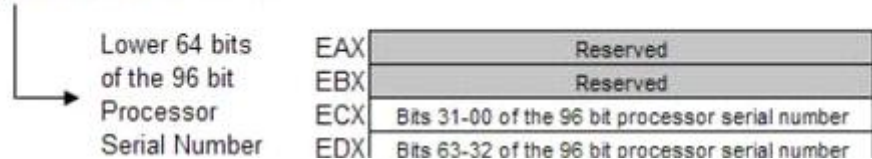
Output of CUID if EAX = 0



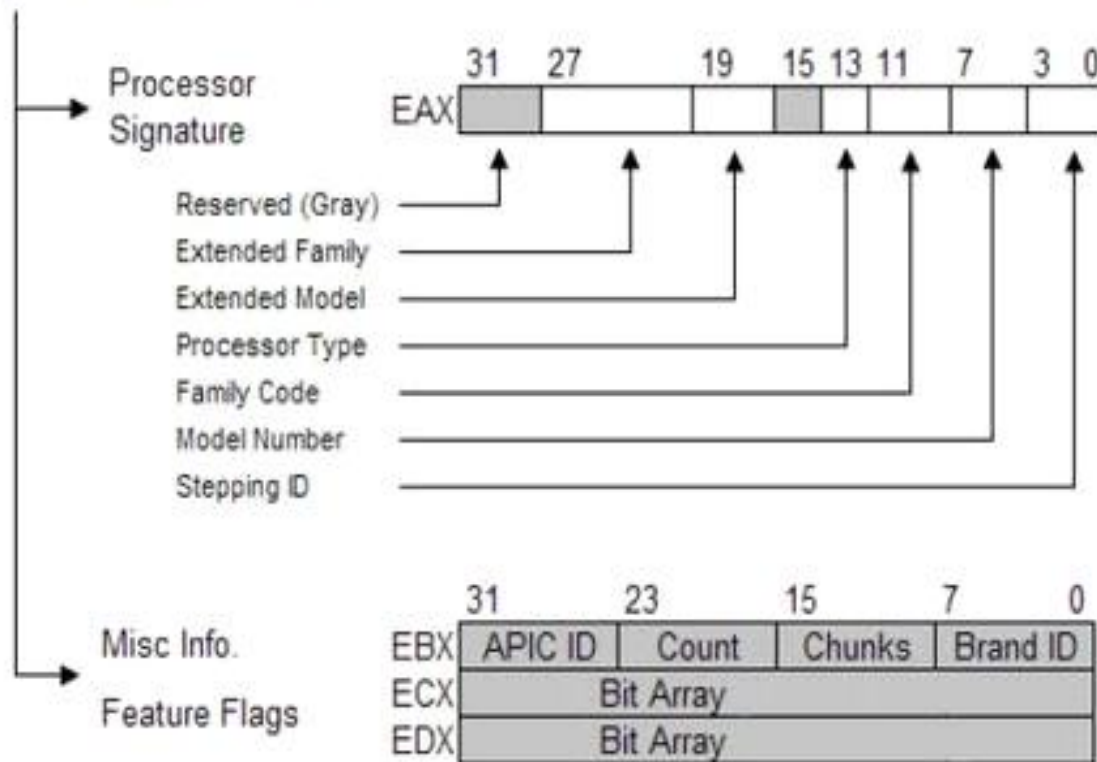
Output of CUID if EAX = 2



Output of CUID if EAX = 3



Output of CUID if EAX = 1



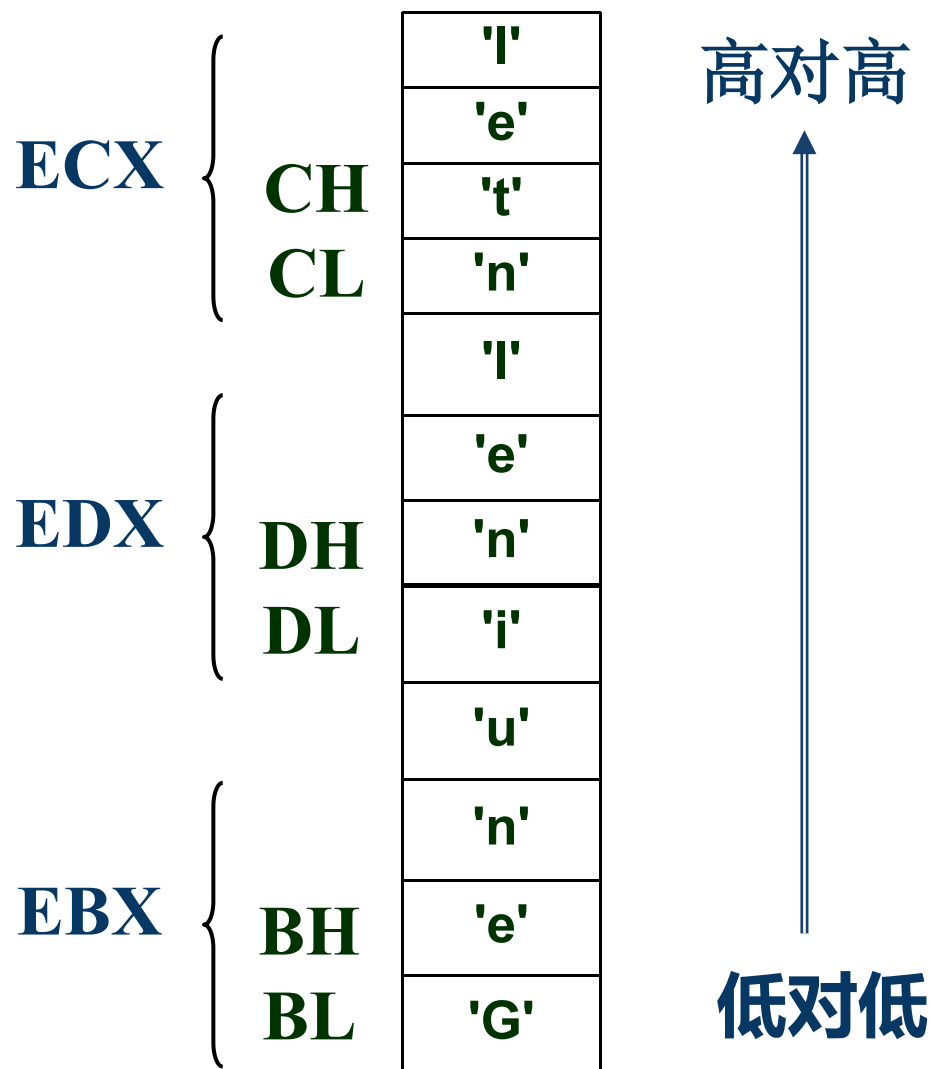
CUID指令

eax=0时CPUID指令的返回结果

- 12个字符
- 以ASCII码形式
- 保存于3个32位寄存器

intel®

GenuineIntel



〔例4-19〕 源程序



```
buffer: .data
        .ascii "The processor vendor ID is "
        .space 12 , 0
        bufsize= .- buffer
        .asciz "\n"
        .text
        mov eax,0
        cpuid          #执行处理器识别指令
        mov dword ptr buffer[rip+bufsize-12],ebx
        mov dword ptr buffer[rip+bufsize-8],edx
        mov dword ptr buffer[rip+bufsize-4],ecx
        lea rax,buffer[rip]    #显示信息
        call dispmsg
```

GenuineIntel

The processor vendor ID is GenuineIntel

〔例4-20〕 查看状态标志



```
.text  
pushf #标志寄存器进栈
```

```
mov eax, dword ptr [rsp]  
call disphd #显示标志寄存器的值  
call dispctrlf
```

```
mov eax, dword ptr [rsp]  
and eax, 1 #截取CF, D0  
call dispuid  
call dispctrlf
```

```
mov eax, dword ptr [rsp]  
shr eax, 6  
and eax, 1 #截取ZF, D6
```

```
call dispuid  
call dispctrlf
```

```
mov eax, dword ptr [rsp]  
shr eax, 7  
and eax, 1 #截取SF, D7  
call dispuid  
call dispctrlf
```

```
mov eax, dword ptr [rsp]  
shr eax, 11  
and eax, 1 #截取OF, D11  
call dispuid  
popf #恢复标志寄存器的值
```

〔例4-21〕 将AL中的值转换成两位十六进制字符串



```
.data #定义数据段  
msg: .byte 0,0,0  
table: .ascii "0123456789ABCDEF"
```

```
.text #定义代码段  
.globl main  
main: # 程序执行起始位置  
lea rbx, table[rip]
```

```
mov al, 100 #0x64  
mov ah, al
```

输出: 64

```
and al, 0xf0  
shr al,4  
movzx rcx, al  
mov al, [rbx+rcx]  
mov msg[rip],al
```

```
mov al, ah  
and al, 0xf  
movzx rcx, al  
mov al, [rbx+rcx]  
mov msg[rip+1],al
```

```
lea rax, msg[rip]  
call dispmsg
```



转移指令

转移指令



- 程序代码在代码段
 - CS: 指明代码段在主存中的段基地址
 - RIP/EIP/IP: 给出将要执行指令的偏移地址
- 程序顺序执行, 处理器自动增量RIP
- 程序控制转移, RIP随之改变
- 程序转移到另外的代码段, RIP和CS都改变
- 控制转移类指令: 改变RIP (有些也改变CS), 即改变程序执行顺序
 - 无条件转移指令
 - 条件转移指令

课堂练习（指令执行顺序）



- 多选题：X86处理器用来指示程序中指令位置的寄存器是（）
A. DS B. CS C. RAX D. RIP
- 判断对错：程序顺序执行，每执行一条指令，指令指针RIP就加1

两条基本的转移指令

- 程序顺序执行，RIP自动指向下一条指令
- 转移指令改变程序执行顺序，即改变RIP[+CS]

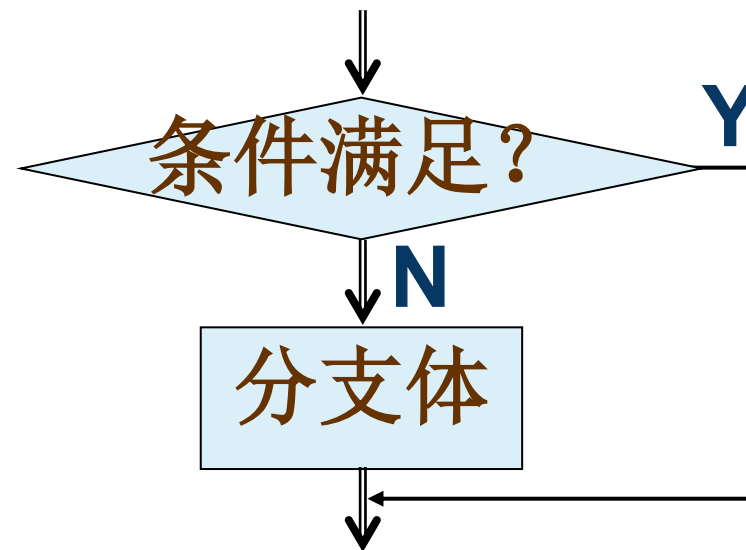
JMP label #无条件转移指令

- 类似C/C++的goto语句

Jcc label #有条件转移指令

- 类似C/C++的if语句

Jcc: jump with condition



1. JMP指令

- 无条件转移：程序无条件改变执行顺序
- JMP指令相当于高级语言的goto语句

JMP 目标地址 #程序转向目标地址，不影响标志位

- JMP指令分类
 - 段内转移：转移指令和目标地址在同一代码段内，只改变RIP
 - 近转移：转移指令和目标地址之间的距离超过8位有符号数范围
 - 短转移：转移指令和目标地址之间的距离在-128~127之间
 - 段间转移：转移指令和目标地址不在同一个代码段内，改变CS和RIP
 - 远转移
 - 任务转移：转移指令与目标地址位于不同的任务
 - 仅适用于保护方式

指令寻址方式

- 相对寻址方式（相对转移）
 - 提供目标地址相对于当前指令指针RIP的位移量
 - 目标地址（转移后的RIP） = 当前RIP + 位移量
 - 相对寻址都是段内转移，最常用、最灵活
- 绝对寻址方式（间接寻址，绝对转移）
 - 指示寄存器或存储单元
 - 目标地址来自寄存器或存储单元、间接获得
 - 寄存器间接寻址：用寄存器保存目标地址
 - 存储器间接寻址：用存储单元保存目标地址
- 直接寻址方式
 - 直接提供目标地址
 - 目标地址（转移后的CS和EIP） = 指令操作数

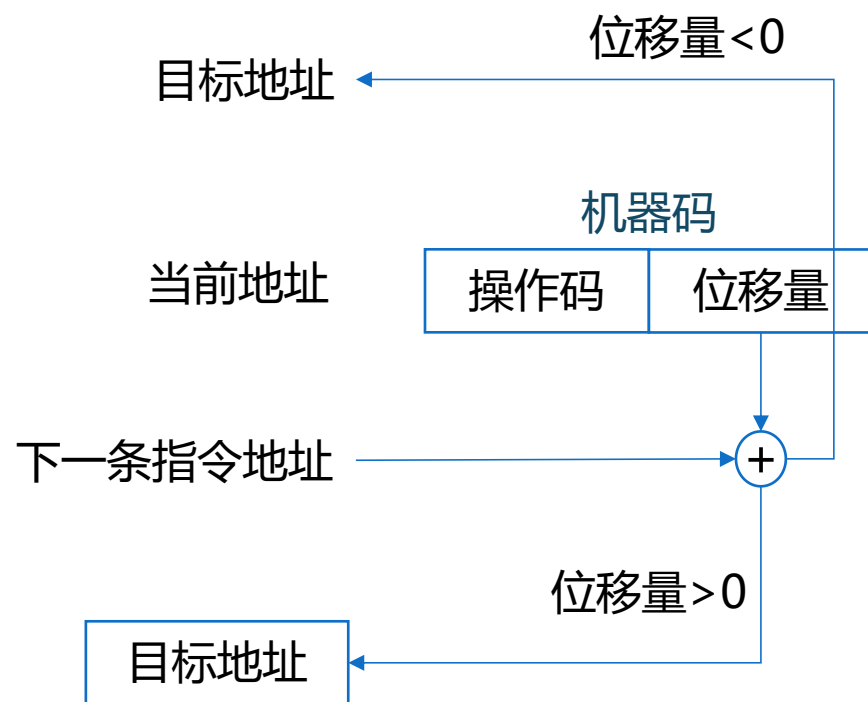
近转移

远转移

(1) 相对转移



JMP label $\#RIP \leftarrow RIP + \text{位移量}$



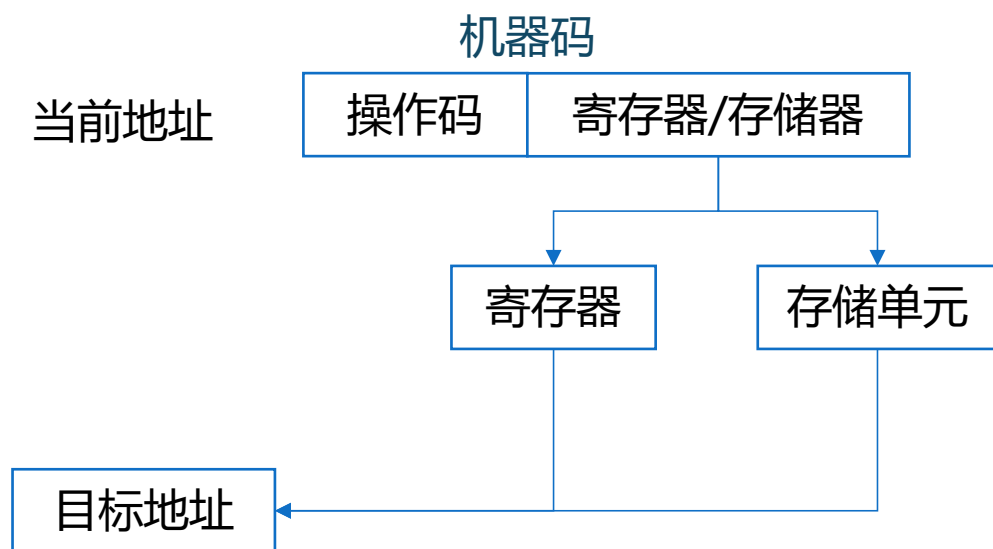
例5-1:

```
.....  
                                .L1:  
0008  C3                        ret  
  
.....  
000d  EBF9                    jmp .L1  
000f  B8000000                 mov eax, 0  
  
.....  
0014  EB0D                    jmp .L6  
0016  .....  
  
.....  
0023  83F802                   .L6: cmp eax, 2  
.....
```

(2) 绝对转移



JMP r16/m16 #IP←r16/m16, 实地址方式
JMP r32/m32 #EIP←r32/m32, 32位保护方式
JMP r64/m64 #RIP←r64/m64, 64位模式



#64位模式

lea rdx, L1[rip]

jmp rdx #RIP←rdx

jmp qword ptr [rdi] #RIP←[rdi]

#32位保护方式

jmp dword ptr [edi] #EIP←[edi]

jmp edx #EIP←edx

#实地址方式

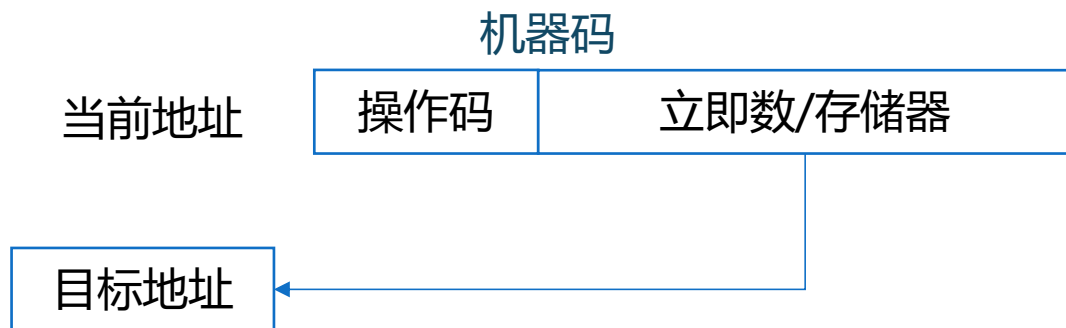
jmp bx #IP←bx

jmp word ptr [di] #IP←[di]

(3) 远转移



JMP ptr 16:16 #IP←16位偏移地址, CS←16位段地址, 实地址方式
JMP ptr 16:32 #EIP←32位偏移地址, CS←16位段地址, 32位保护方式
JMP m16:16 #IP←[mem], CS←[mem+2], 实地址方式
JMP m16:32 #EIP←[mem], CS←[mem+4], 32位保护方式
JMP m16:64 #RIP←[mem], CS←[mem+8], 64位模式



测试题



- 已知nvar是一个变量，指令“jmp nvar”所采用的指令寻址方式是：
 - A 相对寻址
 - B 直接寻址
 - C 存储器间接寻址
 - D 寄存器间接寻址

答案：C

条件转移指令

- 根据指定的条件确定程序是否发生转移
Jcc label ;条件满足, 发生转移; 否则, 顺序执行下条指令
- LABEL表示目标地址, 采用相对转移
 - X86-64处理器64位模式: 8位/32位位移量扩展至64位
 - 32位IA-32处理器: 达到32位的全位移量
 - 16位80x86处理器: -128 ~ +127间的短转移
- 条件转移指令不影响标志, 但要利用标志
- cc表示判断的条件, 包括三类
 - 单个标志状态作为条件
 - 两数大小关系作为条件
 - RCX、ECX是否为0

转移条件cc: 单个标志状态

JZ/JE	ZF=1	Jump if Zero/Equal
JNZ/JNE	ZF=0	Jump if Not Zero/Not Equal
JS	SF=1	Jump if Sign
JNS	SF=0	Jump if Not Sign
JP/JPE	PF=1	Jump if Parity/Parity Even
JNP/JPO	PF=0	Jump if Not Parity/Parity Odd
JO	OF=1	Jump if Overflow
JNO	OF=0	Jump if Not Overflow
JC	CF=1	Jump if Carry
JNC	CF=0	Jump if Not Carry



转移条件cc: 两数大小关系

JB/JNAE	CF=1	Jump if Below/Not Above or Equal
JNB/JAE	CF=0	Jump if Not Below/Above or Equal
JBE/JNA	CF=1或ZF=1	Jump if Below/Not Above
JNBE/JA	CF=0且ZF=0	Jump if Not Below or Equal/Above
JL/JNGE	SF≠OF	Jump if Less/Not Greater or Equal
JNL/JGE	SF=OF	Jump if Not Less/Greater or Equal
JLE/JNG	ZF≠OF或ZF=1	Jump if Less or Equal/Not Greater
JNLE/JG	SF=OF且ZF=0	Jump if Not Less or Equal/Greater

转移条件cc: RCX、ECX是否为0

- JECXZ指令对ECX寄存器进行测试，当ECX=0时，转移到目标地址处。
- JRCXZ指令对RCX寄存器进行测试，当RCX=0时，转移到目标地址处。

JECXZ/JRCXZ label

#ECX/RCX=0，转移；否则顺序执行

CMP ECX/RCX, 0
JZ label

课堂练习（条件转移指令）



- 条件转移指令JC发生跳转的条件是：

A SF=1 B PF=1 C OF=1 D CF=1

- 判断两个数相等则跳转，最对应的条件转移指令是：

A JS B JE C JO D JP



设置条件

怎样产生条件

- 常用指令1：比较指令CMP
 - 进行减法运算
 - 用于判断两个数据大小、是否相等
- 常用指令2：测试指令TEST
 - 进行逻辑与运算
 - 用于判断某位为0或为1等
- 其他指令：能够影响状态标志的指令
加减运算指令、逻辑运算指令、移位指令等

实例1



例：判断两数是否相等，即AL中的数据是否等于0xa：

```
cmp al, 0xa
```

```
jz  L1      #相等 (al=0xa) 转移到L1标号处
```

```
.....
```

```
L1:
```

例：判断是否为大写字母，即AL中D5位是否为0：

```
test al, 0x20 #测试al中的D5位
```

```
jz  cover    #al中的D5=0转移到convert处
```

```
.....
```

```
conver: or al, 0x20    #大写字母变小写字母
```

实例2



例：求两数差的绝对值

```
sub eax, ebx #eax=eax-ebx
```

```
jns L1      #若eax-ebx是正数 (SF=0) , 跳转到L1
```

```
neg eax     #若eax-ebx是负数 (SF=1) , 求补
```

L1:

例：判断两个有符号数X、Y的相加结果是否溢出

```
add ax, bx
```

```
jo  overflow
```

.....

#没有溢出，结果正确

overflow:

#有溢出，进行处理

实例3



#设置校验位

```
and al,0x7f
```

#最高位清零，同时判断“1”的个数

```
jnp next
```

#个数已为奇数，则转向next

```
or al,0x80
```

#否则，最高位置“1”

```
next: .....
```

#判断eax的D1位是否为0

```
ror eax, 2
```

```
jnc L2
```

eax的D1位为0 (CF=0) , 跳转

```
...
```

eax的D1位不为0 (CF=1)

```
L2: rol eax, 2
```

恢复eax

实例4



#比较有符号数大小, $eax=0x0000000f$, $ebx=0x8000000a$

cmp eax,ebx

jge L1 #eax大于等于ebx, 转至L1, 同jnc L1

mov eax, ebx #eax小于ebx, 将ebx值送给eax

L1: #eax=0x0000000f

#比较无符号数大小, $eax=0x0000000f$, $ebx=0x8000000a$

cmp eax,ebx

jae L1 #eax大于等于ebx, 转至L1, 同jnc L1

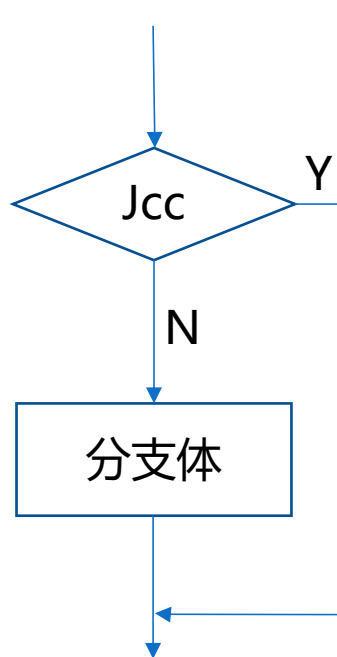
mov eax, ebx #eax小于ebx, 将ebx值送给eax

L1: #eax=0x8000000a

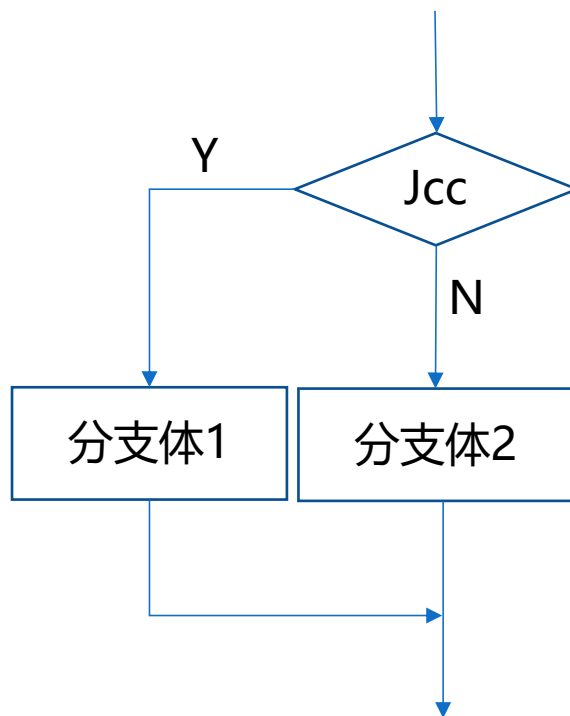


分支结构设计

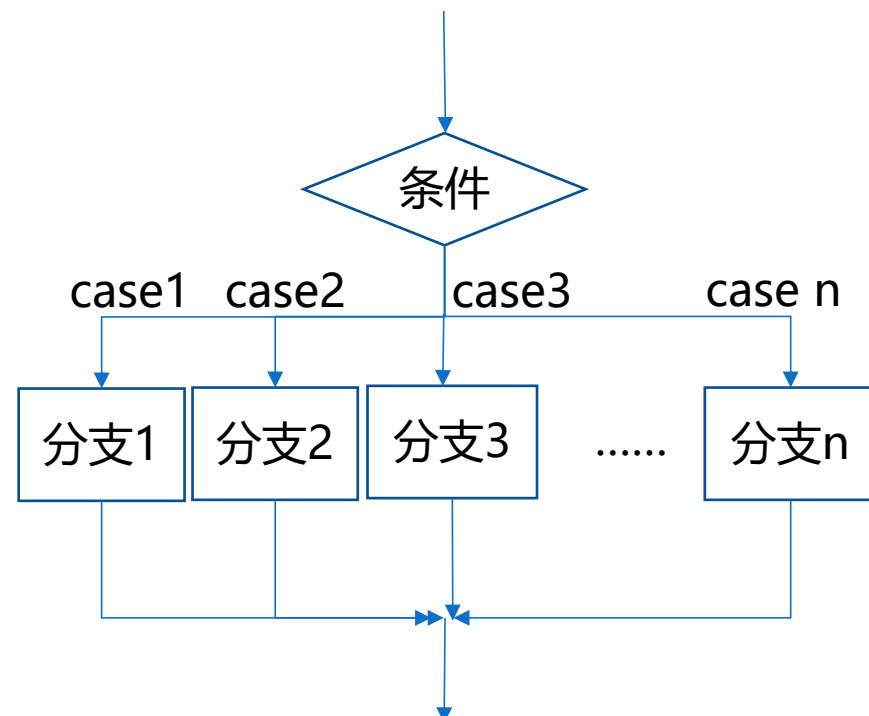
三种分支结构



(a)单分支



(b)双分支



(c)多分支

例5-1: diffabs函数



```
int diffabs(int x, int y){
int t;
if(x>=y)
{
t=x-y;
goto L1;
}
t=y-x;
L1:
return t;
}
```

```
diffabs: #双分支
#ecx: x, edx: y
    cmp ecx, edx
    jge .L4
    mov eax, edx
    sub eax, ecx
    .....
.L1: #汇合
    ret
.L4:
    mov eax, ecx
    sub eax, edx
    jmp .L1
```

diffabs: #单分支

.....
mov eax, ecx
sub eax, edx

cmp ecx, edx
jge .L4
mov eax, edx
sub eax, ecx

.....
.L4:
mov eax, ecx
sub eax, edx
.L1: ret